

Composite Pattern - Organization Management System

Introduction

> The Composite Pattern is a structural design pattern used to organize objects into tree-like hierarchical structures.

It allows both individual objects and groups of objects to be treated in the same way through a common interface.

This pattern is commonly used in systems that contain parent-child relationships such as:

- File systems
- Company organizational structures
- Menus and user interfaces

In this project, the Composite Pattern is implemented using an organization management system that contains departments and employees, where a department can contain employees as well as other departments.

Why We Use Composite Pattern

> The Composite Pattern is used to simplify working with hierarchical structures in a flexible and organized way.

It helps reduce code complexity by allowing developers to treat individual objects and groups uniformly.

It also helps in:

- Improving code organization
- Making the system easier to extend
- Reducing duplicated code
- Improving scalability and maintainability

This pattern is commonly used in applications such as file systems and company organizational structures

Java Code

```
import java.util.ArrayList;
import java.util.List;

interface OrganizationComponent {
    void showDetails();
}

class Employee implements OrganizationComponent {
    private String name;
    private String position;

    public Employee(String name, String position) {
        this.name = name;
        this.position = position;
    }

    @Override
    public void showDetails() {
        System.out.println("Employee: " + name + " | Position: " + position);
    }
}

class Department implements OrganizationComponent {
    private String name;
    private List<OrganizationComponent> members = new ArrayList<>();

    public Department(String name) {
        this.name = name;
    }

    public void add(OrganizationComponent component) {
        members.add(component);
    }

    public void remove(OrganizationComponent component) {
        members.remove(component);
    }

    @Override
    public void showDetails() {
        System.out.println("Department: " + name);

        for (OrganizationComponent component : members) {
            component.showDetails();
        }
    }
}

public class Main {
    public static void main(String[] args) {

        Employee emp1 = new Employee("Ahmed", "Developer");
        Employee emp2 = new Employee("Sara", "Designer");
        Employee emp3 = new Employee("Ali", "Security Engineer");
        Employee emp4 = new Employee("Lina", "HR Manager");

        Department securityTeam = new Department("Security Team");
        securityTeam.add(emp3);

        Department itDepartment = new Department("IT Department");
        itDepartment.add(emp1);
        itDepartment.add(emp2);
        itDepartment.add(securityTeam);

        Department hrDepartment = new Department("HR Department");
        hrDepartment.add(emp4);

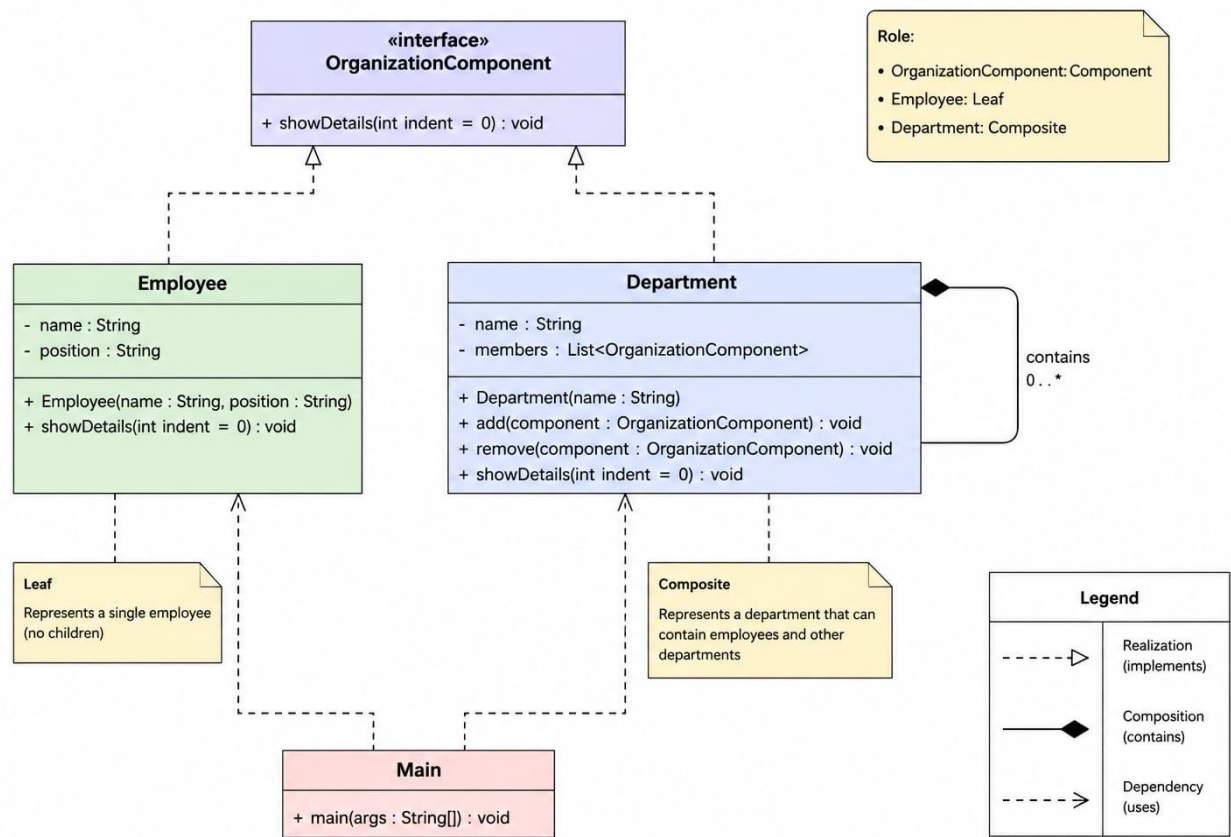
        Department company = new Department("Company");

        company.add(itDepartment);
        company.add(hrDepartment);

        company.showDetails();
    }
}
```

System Diagram

Composite Pattern - Organization Management System



Conclusion

The Composite Pattern helps developers build flexible and organized systems using a common interface. It makes hierarchical structures easier to manage and extend.

Group Names

- Hiba
- Tasneem
- Noor